OS Orange Problem 2

Course Code : UE24CS242B

Subject : Operating Systems

Section : 4J

SRN : PES2UG24CS570

Name : Vaishnavi Bandaru

GitHub Repo Link : https://github.com/vaishnavib04/PES2UG24CS570-pes-vcs/tree/main

Screenshots:

Phase 1:

1 A

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1 B

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
seed@sever2:~/PES2UG24CS570-pes-vcs$
```

Phase 2:

2A

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
seed@sever2:~/PES2UG24CS570-pes-vcs$
```

2B

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ find .pes/objects -type f
seed@sever2:~/PES2UG24CS570-pes-vcs$
```

Phase 3:

3A

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
seed@sever2:~/PES2UG24CS570-pes-vcs$ echo "hello" > a.txt
seed@sever2:~/PES2UG24CS570-pes-vcs$ ./pes add a.txt
seed@sever2:~/PES2UG24CS570-pes-vcs$ ./pes status
Staged changes:
  staged:     a.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  pes.c
  untracked:  test_objects.c
  untracked:  tree.c
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  README.md
  untracked:  .gitignore
  untracked:  commit.h

seed@sever2:~/PES2UG24CS570-pes-vcs$
```

3B

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ cat .pes/index
33188 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776436100 6 a.txt
seed@sever2:~/PES2UG24CS570-pes-vcs$
```

Phase 4:

4A

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ ./pes log
commit 3f242002468b86d559ae923fd05a945d90380052fd6304871ef53ae8ab124cf9
Author: PES User <pes@localhost>
Date:   1776447082

    Add farewelldd fQ

commit 16fa932e12db9ae978036ece72cd2561c177b593dfb08639e2221f2b75667ebf
Author: PES User <pes@localhost>
Date:   1776447066

    Add worldelldd fQ

commit 92c4422ecd65ea5c47791eb67ffdabf8ac6662d92d0cb41097365b1b3ad46762
Author: PES User <pes@localhost>
Date:   1776447054

    Initial committial commiQ
```

4B

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/16/fa932e12db9ae978036ece72cd2561c177b593dfb08639e2221f2b75667ebf
.pes/objects/39/c3f4db583d4d919f49897782f2a71fbdc42f36ab1a2c998a5b0b9036100645
.pes/objects/3f/242002468b86d559ae923fd05a945d90380052fd6304871ef53ae8ab124cf9
.pes/objects/4b/5297089f9537660c5a3cc9fac5ff31af5458737d7bd3a89b772c56fda55e07
.pes/objects/92/c4422ecd65ea5c47791eb67ffdabf8ac6662d92d0cb41097365b1b3ad46762
.pes/objects/9f/2a7f3b00f22334f6adc2721fb1f89cd969f02264d37cf6a2c32ba09beb6822
.pes/refs/heads/main
```

4C

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ cat .pes/refs/heads/main
3f242002468b86d559ae923fd05a945d90380052fd6304871ef53ae8ab124cf9
seed@sever2:~/PES2UG24CS570-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
seed@sever2:~/PES2UG24CS570-pes-vcs$
```

Final - Full integration test screenshots:

```
seed@sever2:~/PES2UG24CS570-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 1180be37d6bb... Initial commit

Log after first commit:
commit 1180be37d6bb52ce0becf74944ca986fe407d4c489edcb6946ce208de7b709d3
Author: PES User <pes@localhost>
Date:   1776447094

    Initial committial commiQ


--- Second Commit ---
Committed: 79594954103b... Update file.txt

--- Third Commit ---
Committed: c96e796e7f66... Add farewell

--- Full History ---
commit c96e796e7f663309d674459ea706842ca1a0e26381fd9a19f00e33d719d0b69e
Author: PES User <pes@localhost>
Date:   1776447094

    Add farewelldd fQ

commit 79594954103b9f7a0650a0874f73ee1b852273e0427c21d00eaa2d0ca053308e
```

```
commit 1180be37d6bb52ce0becf74944ca986fe407d4c489edcb6946ce208de7b709d3
Author: PES User <pes@localhost>
Date:   1776447094

    Initial committial commiQ


--- Second Commit ---
Committed: 79594954103b... Update file.txt

--- Third Commit ---
Committed: c96e796e7f66... Add farewell

--- Full History ---
commit c96e796e7f663309d674459ea706842ca1a0e26381fd9a19f00e33d719d0b69e
Author: PES User <pes@localhost>
Date:   1776447094

    Add farewelldd fQ

commit 79594954103b9f7a0650a0874f73ee1b852273e0427c21d00eaa2d0ca053308e
Author: PES User <pes@localhost>
Date:   1776447094

    Update file.txtfQ

commit 1180be37d6bb52ce0becf74944ca986fe407d4c489edcb6946ce208de7b709d3
Author: PES User <pes@localhost>
Date:   1776447094

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c96e796e7f663309d674459ea706842ca1a0e26381fd9a19f00e33d719d0b69e

--- Object Store ---
Objects created:
6
.pes/objects/11/80be37d6bb52ce0becf74944ca986fe407d4c489edcb6946ce208de7b709d3
.pes/objects/79/594954103b9f7a0650a0874f73ee1b852273e0427c21d00eaa2d0ca053308e
.pes/objects/9f/2a7f3b00f22334f6adc2721fb1f89cd969f02264d37cf6a2c32ba09beb6822
.pes/objects/bc/eb6654d7dc48c07e58576d92004b4ef7e68877fc35fea9d41a913a36953590
.pes/objects/c9/6e796e7f663309d674459ea706842ca1a0e26381fd9a19f00e33d719d0b69e
.pes/objects/fc/b3033c1ab6957a33ba378b667b8b96f9d487e51659f80f33a1ee641f5f4c9e

=== All integration tests completed ===
```

Phase 5 and 6:

5.1

To implement checkout, .pes/HEAD must be updated to point to the target branch reference in .pes/refs/heads/.

Then the commit hash from that branch is read, and its tree is loaded from the object store. The working directory is updated to match this tree by:

- deleting files not in the tree
- creating missing files
- updating modified files

This is complex because it must keep working directory, index, and object store consistent without losing data.

5.2

A dirty working directory is detected by comparing index entries with working directory files.

For each tracked file:

- If missing , deleted
- If size/mtime/hash differs , modified

If any tracked file has changes that differ from the target branch tree, checkout is blocked to prevent overwriting uncommitted changes.

5.3

Detached HEAD means HEAD points directly to a commit instead of a branch.

Commits made in this state are not attached to any branch and may become unreachable.

They can be recovered by finding the commit hash and creating a new branch pointing to it.

6.1

Garbage collection removes unreachable objects from the object store.

First, all branch references are used as starting points. From these, all reachable commits are traversed, and recursively their parent commits, trees, and blobs are marked as reachable. A hash set is used to store all reachable object hashes efficiently.

After marking, the object store is scanned and any object not in the reachable set is deleted.

For a repository with around 100,000 commits and 50 branches, the number of visited objects is roughly proportional to the reachable history. Since branches share history, the actual traversal is usually less than the total commits and may include a few hundred thousand objects including trees and blobs.

6.2

GC and commit conflict if GC deletes objects while a commit is creating them.

Example: GC marks objects as unreachable before a commit finishes writing them, causing commit failure.

Git avoids this using locking and atomic updates so GC only runs on a consistent snapshot where no commit is writing objects.